

Question	Answer
what is weak reasoning in this context and how it impact good prompt	Weak reasoning means the model jumps to an answer without properly thinking through the steps. In this example, it might promise a refund straight away without first asking for the order number or checking whether the order is eligible. A good prompt fixes this by guiding the model's thinking, for example: "First check if the order number is given, then check refund eligibility, then suggest next steps." Clear steps lead to more correct and reliable answers.
Is there something like a prompt evaluator to assess if a given prompt is constructed well? Can we assume that a prompt that works well on claude will work as effectively on GPT or gemini etc.?	Yes. Tools like promptfoo, LangSmith, and OpenAI Evals let you test a prompt on a set of sample inputs and score the outputs, often with another LLM acting as a judge. A prompt is judged well built by how well it performs on your test cases, not by how it looks. No, we can't assume that. Each model is trained differently, so the same prompt can behave differently on Claude, GPT, or Gemini. That's why the slide stresses testing: always re-test the prompt on each model and adjust it as needed.
How do we actually train the model on how should the output look like?	Prompt Optimisation does not mean updating the weights of the model but to modify the prompt such that we can get the desired output from the model.
what is semantic versioning	Semantic versioning is a way of numbering versions as MAJOR.MINOR.PATCH, so the number itself tells you how big a change was.

As we have so many different types of prompts and prompt engineering is in place, all the big tech companies like Google or OpenAI, are they charging based on prompt type? For example, a pro version will have best prompt aligned and for non-paid versions , prompts are not much engineered? Becuase model performance is different .	No they are not charging basaed on the prompt type but on the number of tokens that the prompt contains. The longer the length of the prompt, the greater is the charge.
Are there any standard packages available for writing regression, semantic, etc testing of the prompts?	promptfoo, DeepEval, and Ragas (for RAG apps), which work like unit tests for prompts. You keep a set of test inputs with expected outputs, then check the results by exact match, semantic similarity (meaning-based), or an LLM judge. LangSmith and OpenAI Evals also support running the same tests on every new prompt version to catch regressions.
By looking at this , can there be a new field in Engineering called "Prompt testing/Engineering" ?	This is already happening in the industry. It started and we called "Prompt Engineering" (which many initially mistook for just writing clever prompts) Today, this falls under AI Engineering or LLMOps. Just like traditional software engineering has QA (Quality Assurance) and DevOps to test and deploy code, building robust AI applications requires engineers to design automated evaluation pipelines, maintain regression sets, and manage prompt versioning. As AI systems become more complex and autonomous,

are we talking about versions of LLM or the models for which prompts are submitted	Here we're versioning the prompts themselves, not the LLM. The model stays the same, and each time you change the prompt text, you save it as a new version (v1.0.0, v1.1.0, and so on) so you can compare results and roll back if needed. It's still good practice to note which model each prompt version was tested on.
user is asking the prompt, why we need to store the prompt(product developer)	Here "prompt" means the instructions the product developer writes behind the scenes- system prompt, like the ShopNow rules, not the question the end user types. The developer stores and versions these instructions because they control how the app behaves for every user. If a change makes answers worse, the team can compare versions, find what broke, and roll back quickly.
How do we test the prompts for optimize? In normal application testing, there are test cases and expected outputs. How does this work for prompts? Do we define expected outputs. The output may vary based on model parameters, temprature etc	Yes, but for LLM prompts we usually do not require one exact expected output. Instead, we define evaluation criteria or acceptable output properties. For example, factual correctness, required fields, relevance, tone, or whether certain information is present.
whats version control tools are used in case of LLM versioning ?	Depending on whether you are versioning the prompts or the model weightts For Prompt versioning, Git/Github or Dedicated tools like LangSmith (Prompt Hub), PromptLayer, Weights & Biases (Weave). For LLM weights, Hugging Face Hub, MLflow & DVC (Data Version Control)

What is prompt chaining and role playing?	Prompt chaining means breaking a big task into smaller steps, where the output of one prompt becomes the input to the next. For example, the first prompt pulls out the order details, and the second prompt uses them to write the refund reply. Role playing means telling the model to act as a specific person or expert, like "You are a customer support assistant for ShopNow." This shapes its tone, focus, and style of answering.
are these user prompts or system prompts? We cannot expect all users are learned prompt engineering?	These examples are user prompts. You're right, we can't expect every user to learn prompt engineering. That's why developers use the same ideas in the system prompt, which adds clarity, format, and limits behind the scenes. Some apps also quietly rewrite a vague user question into a clearer one before sending it to the model, so users get good answers even with simple questions.

Does the prompt structure depend on the type of model being used? For example, the prompt structure for Gemini v/s GPT-Sol. If yes, then how can we determine what is the best and most optimized structure of the prompt that needs to be used?	Yes, Prompt structure depends on the underlying model because each has different default behaviors, context handling, and formatting preferences. To get the most optimized structure, you can do manual trial and error, but it will take some time. You can shift away from manual trial and error and use Automated Prompt Optimization frameworks like DSPy. Rather than hand-crafting the prompt, you define a declarative "Signature" (specifying the exact inputs and outputs), provide a small set of training examples, and set an evaluation metric. The framework's optimizer will then algorithmically generate, test, and refine thousands of structural variations on your specific model until it mathematically maximizes the evaluation score.
is this optimozation applies for System,User and Assitantm prompts also? are there any guidlines ?	Yes, the same ideas apply to all three. The system prompt is where optimization matters most, since the developer controls it: role, rules, format, and limits. User prompts improve with clear, specific requests. Assistant messages can hold sample ideal replies (few-shot examples) that show the model how to answer. For guidelines, OpenAI, Anthropic, and Google each publish free prompt engineering guides in their official documentation.

Hi sir..Do we preserve context layers ,prompts and responses if we have chain prompts...and if we preserve it what is that database except md files	Yes, it's good practice to save each step of a chain (the prompt sent, the context, and the response) so you can debug, test, and reuse it. Strictly coming from how I have used - apart from md files, common choices are tracing tools like LangSmith or Langfuse, which record every step automatically. Regular databases like PostgreSQL or MongoDB can store them as JSON records. Redis is useful for short-term session memory, and vector databases like Chroma or Pinecone help when you need to search past context by meaning.
How the stages in chain prompts are defined, like model knows in how many prompts it has to break a big prompt considering it is not breaking the full meaning of the prompts or questions asked in the prompt	In standard prompt chaining, the model itself does not automatically decide how to break down a large prompt. Instead, the developer or prompt engineer explicitly defines these stages in their application code. For example, if you are building a research tool, you might code a three-step chain: Prompt A: Extract key entities from a document. Prompt B: Search for those entities to gather facts. Prompt C: Format the gathered facts into a summary.

Is there any difference in how the models understand 20 sentences vs 20 bullet points?	I hope I got your question right, if not please ask again. The model reads both as plain text, so the meaning comes through either way. The difference is clarity. Bullet points separate each instruction clearly, so the model is less likely to miss or mix up rules. Sentences work better when ideas are connected and need explanation, like "do X because of Y." Also, the model often copies your style, so bullets in the prompt can lead to bullets in the answer.
suppose incase of one prompt O/P is an I/P for other prompt,how to optimize the the whole chain ?	First, test and improve each step on its own, since an error in step 1 carries into every later step. Second, make each step's output clean and structured (like JSON or fixed bullets) so the next step reads it correctly. Third, test the whole chain end to end on sample inputs. If the final result is weak, check each step's output to find where it went wrong and fix that step first. I have used tools like DSPy can also optimize a full chain automatically.
while dividing the prompt into multiple subprompts.. the token utilized should be same? isnt it?	Not quite. Chaining usually uses more tokens. Each subprompt needs its own instructions, and the output of one step is sent again as input to the next, so some text gets counted more than once. In the slide example, the translated English text is produced in step 2 and then sent again as input to step 3. The trade-off is higher cost and time for better accuracy and easier debugging.

Chroma and Pincone will be used mostly for RAG based apps..Right sir	Yes, that's right! Chroma and Pinecone are vector databases, mostly used in RAG apps to store documents as embeddings and find the most relevant pieces by meaning. They're also used for long-term chat memory, where past conversations are fetched by similarity.
Can we infuse COT in Propmt Chaining?	Yes, you can do it. Prompt Chaining and Chain-of-Thought (CoT) operate at different levels. Chaining is an architectural pattern where you connect multiple distinct prompts via your application code (Prompt A passes data to Prompt B). CoT is a reasoning technique used inside a single prompt (asking the model to think step-by-step) For example: Step 1 (Prompt A): Extract raw facts from a document (No CoT needed; straightforward). Step 2 (Prompt B): Analyze the facts to identify contradictions (Infuse CoT here to ensure the model reasons deeply). Step 3 (Prompt C): Format the final conclusion (No CoT needed; strict formatting).

doesn't chain prompting actually use more tokens than one single large combined token?	Yes, usually it does. Each step in the chain needs its own instructions, and the output of one step is sent again as input to the next, so some text gets counted twice. In the slide example, the English translation is produced once and then sent again to the next step. The extra tokens buy better accuracy and easier debugging.
How does cost for tokens apply. I get that single long prompt will utilise more tokens but sum of tokens of multiple prompts in prompt chaining will also do same right ? How can we reduce token consumptions to save the total cost ?	You pay for every call, based on the input tokens you send plus the output tokens you get back (output usually costs more). In chaining, you pay for each step, so the total is usually equal to or more than one big prompt. To reduce cost we can try few or all of these techniques depending on use case - - Keep instructions short and clear. - Pass only the needed part of each output to the next step. - Use a smaller, cheaper model for simple steps like translation. - Limit output length. - Use prompt caching when the same system prompt repeats. - Only split steps that really need splitting.

What happens on the LLM when the user assigns a role or persona while prompting? Why is this beneficial?	When you assign a role or persona, you are mathematically constraining the model's output probability distribution. Without a role, a vague prompt places the model in a state of high entropy, causing it to sample from the most generic, average responses in its training data (resulting in bland corporate voice). Adding a role acts as a "projection operator," reducing that big probability space down into a specific subset of domain vocabulary, tone, and style.
Following up on my previous question, are there basic prompt guidelines for auto-regressive v/s auto-encoding family of models? Since the way these models read the prompt are different.	Yes, also this is a good question Darshan. Autoregressive Models (Decoders like GPT, Gemini, Llama), Because they read sequentially from left to right, prompts must be structured chronologically. You place broad system instructions at the top, followed by few-shot examples, and end with the specific query. Autoencoding Models (Encoders like BERT, RoBERTa), They are not designed for text generation. Therefore, you cannot use open-ended instructions or ask them to think step-by-step. you must use Cloze-style prompting (fill-in-the-blank). For example, to classify sentiment, you format the prompt as a pattern: "The food was terrible. Overall, it was a [MASK] experience." The model then calculates the probability of specific tokens fitting into that exact blank.

Is prompt chaining and role playing techniques are opposite to each other because in role playing we are give more tokens in a single prompt as against breaking it down into smaller prompts	No, they aren't opposites. They solve different problems and work well together. Role playing sets who the model should act as, and it usually adds only one short line. Prompt chaining sets how a big task is split into steps. You can combine them by giving each step in the chain its own role, for example "You are a professional translator" for the translation step and "You are a data analyst" for the fact-finding step.
what can be the disadvantages of Assigning the roles while adding prompts	A role can make the model overconfident, sounding like an expert even when it's wrong, and it can narrow answers too much or add unnecessary style and length. It also adds tokens and may not improve accuracy on factual tasks. For example, "You are a senior doctor" may produce confident medical advice that still contains errors.
Do Prompt Optimization methods mainly revolve around System prompt optimization? or, user prompts can also have guided optimization?	Mostly system prompts, because the developer controls them and they run for every user, so improving them helps everyone. Few-shot examples are often optimized too. User prompts can also be optimized. Apps can quietly rewrite a vague question into a clearer one before sending it to the model, offer ready-made templates, or add an "improve my prompt" button that refines what the user typed.

can we add cost handling in the system prompt itself? Cosume less tokens to solve the user query?	Partly, yes. You can add instructions like "Answer in under 100 words" or "Be brief." This reduces output tokens, which usually cost more. But the model can't track its own cost or token count exactly, so for a hard limit, also set the max_tokens setting in the API call. Keep the system prompt itself short too, since it's sent and charged with every call.
What is the best practice to design the number of stages in a prompt. I think having a lot of prompt will lead to some issues. What parameters to check in this case before defining the stages in prompts ?	You're right, too many stages add cost, delay, and more places for errors to pass along. A good practice is to start with one prompt and split only when it fails, for example when the model skips steps or mixes tasks up. Keep only the stages that clearly improve results. Ask - Is it a truly separate task, like translating vs. extracting facts? Does splitting improve accuracy on your test set? The added token cost and response time. Whether the step needs a different model or output format. Whether errors from earlier steps carry forward.
seed prompts are nothing but Prompt Registry?	Not exactly. Seed prompts are the starting prompts that APO begins with. They're written by hand or generated by an LLM, and then improved over many rounds. A prompt registry is a storage system where teams save, version, and manage their prompts. You can pick seed prompts from a registry, and store the final optimized prompt back in it, but the two aren't the same thing.
who is ging to create candidate prompt	Usually another LLM, often called the "optimizer LLM," creates them. It takes the seed prompt and writes new versions by rephrasing it, adding or removing instructions, or fixing mistakes found during evaluation.

When we use claude code/chat GPT other IDEs we cannot set the System prompt and also the role? Do we know how is that configured that it works corect for every type	These tools come with a hidden system prompt written by the company, tuned and tested across many kinds of tasks, so it works well generally but isn't perfect for every case. You usually can't edit it, but you can add your own instructions on top, such as a CLAUDE.md file in Claude Code or "Custom Instructions" in ChatGPT.
We do seeperate evaluations...I believe...evaluations for LLM and evaluations for prompt are differenet	Yes, you're right! In LLM evaluation, the prompt stays fixed and we compare different models, often on general benchmarks like MMLU. In prompt evaluation, the model stays fixed and we compare different prompt versions on our own task's test cases. The tools are similar (test sets, scores, LLM judges), but the thing being tested is different.
if optimizer llm is there why we need manual prompt optimization	APO doesn't replace manual work, it builds on it. It still needs a good starting (seed) prompt, a test set, and a scoring method, and humans usually create all three. APO also runs many LLM calls, so it costs more time and money. For simple tasks, or when you need specific business rules, tone, or domain knowledge, manual tuning is often faster and enough. APO helps most when you have good test data and want to squeeze out extra accuracy.
is exit criteria pre-decide? what is example	Yes, the exit criteria are decided before the loop starts. examples - The best prompt reaches a target score, say 90% accuracy on the test set. - A fixed number of rounds is completed, say 10. - The token or cost budget runs out. The loopstops if one of these met.

How prompt optimization help in token economics? Please help with some important points.	A shorter, clearer prompt cuts input tokens on every call, and clear output instructions stop the model from rambling, which cuts the pricier output tokens. A well-optimized prompt can also let a smaller, cheaper model do the job, and fewer wrong answers mean fewer retries.
If openai or anthropic update their model weights at the backend based, our versioned prompts (optimized for specific tasks that worked well) may regress?	Yes, it happens to me soo many times, This is a well-known industry issue, its called Model Drift or Vendor Drift. When providers like OpenAI or Anthropic update their model weights, sometimes silently without a changelog or version number change, the exact same prompt can suddenly produce degraded outputs, altered formatting, or different reasoning paths.
Can the APO be placed in LLM as Judge?	Yes! In the evaluation step, an LLM can act as the judge, scoring each candidate prompt's answers against a rubric.
so in current LLM Models which we use do they use system prompt generated by LLM and all these APO generated prompts are they system Prompts?	Not necessarily, most production system prompts (Claude's, ChatGPT's) are still largely hand-written and refined by human engineers through testing, not fully LLM-generated. APO methods (like APE, OPRO) use an LLM to search/refine prompt text, but that output is just optimized instruction text — it's not inherently a "system prompt."

Where is APE used and is the evaluation is. also done by LLM and it chooses the best prompt by itself?	APE is used for tasks where you have clear input and output examples, like classification, extraction, translation, or rule-based tasks such as the antonym example. It started as a research method, and its ideas now appear in tools like DSPy. Yes, the evaluation is also done by an LLM. As on the slide, it scores each candidate, for example by how likely the correct output is (log probability) or by accuracy on test examples. It then automatically keeps the highest-scoring prompt. Humans only provide the examples and decide how scoring works.
this scoring is what we do via Rogue or Blue or Bert ?	ROUGE, BLEU, and BERTScore measure output-to-reference similarity (n-gram overlap or embedding similarity), and is useful when you have a gold-standard answer to compare against. Prompt quality scoring (clarity, robustness, consistency) is different. It typically uses LLM-as-judge or task-success metrics (accuracy, pass rate across runs), since there's often no single "correct" reference text.
why does the prabability is showing negative here	It's log probability, not plain probability. A probability is always between 0 and 1, and the log of any number below 1 is negative. So the closer the score is to 0, the higher the probability and the better the prompt. For example, -0.16 is about 85% probability (the winner), while -1.08 is only about 34%.

why are prompt score values in negative range?	It's log probability, not plain probability. A probability is always between 0 and 1, and the log of any number below 1 is negative. So the closer the score is to 0, the higher the probability and the better the prompt. For example, -0.16 is about 85% probability (the winner), while -1.08 is only about 34%.
how the end user will do all these instruction search	End users don't do this at all. Instruction search is done by developers while building the app, usually offline before launch. They run the search, pick the best prompt, and place it inside the app as the system prompt. End users simply type their questions as usual and get better answers without seeing any of this.
How we score the prompts which good scoring and which are bad scoring	We run each candidate prompt on a set of test examples where we already know the correct answer, then give it a score. Common ways to score are Accuracy, Log Probability, LLM Judge. Then we rank all the candidates. The top-scoring ones are treated as good and kept, and the lowest ones are discarded.
How many iterations does it take to generate perfect candidates in APO?	In Automatic Prompt Optimization (APO), there is no fixed number of iterations. Because prompt optimization is a search algorithm across natural language, it continues looping until it hits a developer-defined stopping condition, It can be Target Score, Conergence, or Max Iterations as well.

for instruction search are we using LLM as Judge for scoring LIKELIHOOD?	Not exactly. In likelihood scoring, the LLM doesn't judge or give opinions. We give it the instruction and the input, then simply read how likely it is to produce the known correct output (like "indirect" for "direct"). That probability is calculated directly by the model. LLM as Judge is different: a model reads a generated answer and rates it using a rubric.
We are discussing about the optimization for this activity also solution will use tokens. so how is this solving the problem?	This optimization process happens only once before deploying the LLM. After deployment, the optimized prompt is used which saves token cost.
Are there any packages for the APO?	You're thinking like a true AI engineer, good good. These are all the packages on top of my head DSPy, TextGrad, prompt-ops, AutoPrompt, Promptfoo
What do we do with these list of prompts generated or filtered out at the end of APO do they go as system prompts?	Usually only the final best prompt is used. It goes into the app, most often as the system prompt or task instruction, and is saved as a new version. The next few top prompts can be kept for A/B testing or as seeds for the next round of optimization. The rejected ones are usually just logged for reference and not used.
gradient optimization also APO ?	Yes

The same model as inference model and the same one for scoring; is that a limitation as the model is pre-trained on same parameters and will carry the flaws/biasness in the model. Should there be a red teaming (not for attack but for critical evaluation) done like GPT evaluated by Claude models?	Great point, and yes, this is a real limitation. A model can prefer its own style and miss its own blind spots, which is known as self-preference bias. Suggested fixes are - Use a different model as the judge, for example Claude checking GPT outputs, Score against labeled correct answers wherever possible, since accuracy doesn't depend on the model's opinion, and human review.
Will the optimization and evaluation be impacted if there are too many spelling mistakes?	Yes, Spelling mistakes can hurt both stages. During optimization, they add noise the LLM has to parse around, sometimes shifting attention or triggering misinterpretation of intent. During evaluation, if you're using embedding-based metrics (BERTScore) or LLM-as-judge, typos can lower similarity scores or make the judge model penalize clarity/quality even if intent was clear.
how can we assign some evaluation in automatic optimization technique to check if the prompts are following the right order or they are not going out of context, in case of out of context then reset the conversation and start from scratch ?	In APO, you can add composite metrics (like regex for step ordering) that penalize and discard drifting prompts. For runtime, drift is handled in your orchestration code. When an evaluator detects drift, a circuit breaker intercepts the flow to reset the context by clearing the conversation history and starting fresh.
Gradient here is like feedback ?	Yes, gradient refers to the reasoning behind the flaws in the initial prompt. It acts like a feedback to improve the prompt.

in Gradient Optimizaton, if the response is not a label but natural language free form response. Then how gradients are evaluated and Which Loss is used for this gradient?	There's no numeric loss or real gradient here. For free-form answers, an LLM judge compares each response with a reference answer or a rubric, gives a score, and writes what went wrong. That written critique is the "gradient." The score, from the judge or from semantic similarity to the reference, acts like the loss and is used to pick the best prompt.
so are gradients helping to find flaws and then you are changing the prompt accordingly	Yes, exactly. The gradients point out the flaws in the initial prompt and help in improving the prompt.
Which loss function used in gradient Optimizations? how to know which one is used? and also used activation funtion in it?	In ProTeGi there's no loss function or activation function, because no model weights are trained and the LLM stays frozen. It uses a task metric, like accuracy or F1 on a batch of examples, to find mistakes and pick the best prompt. To know which metric is used, check the evaluation section of the paper or tool. Activation functions exist only inside the LLM and aren't changed.
How to find out the gradient of a prompt	To find the gradient of a prompt, you do not use math or calculus. Instead, you use an LLM as an evaluator. You feed the current prompt and its failed test outputs into an LLM, and ask it to write a critique explaining exactly why the prompt failed. That resulting natural language critique is your "textual gradient."
Do we Give some prompt to LLM to cal. gradient on initial prompt. If yes, How does we know the prompt we gave the LLM to act as gradient calculator is highly accurate.	Yes .You give the LLM a meta-prompt asking it to critique the current prompt and suggest an edit, functioning as a "textual gradient." There's no guarantee this meta-prompt is accurate. It's validated empirically, not proven, you check if the optimization loop actually improves task performance over iterations, run ablations on different meta-prompt phrasings, and often spot-check with human review.

how gradient based optimization takes place in code? Does it persist post session as well?	In code, it's a loop of LLM calls: run the prompt on a batch, collect the wrong answers, ask the LLM to explain the errors (the gradient), ask it to rewrite the prompt, score the new prompts, keep the best, and repeat. Libraries like TextGrad and DSPy do this for you. Nothing persists automatically, because the LLM itself doesn't learn. You save the final prompt to a file or prompt registry and use it later.
how do selection algorithms work	Testing every candidate on the full dataset is costly, so bandit selection saves effort. Each candidate prompt is first tested on a few examples. Promising ones get more tests, while weak ones are dropped early. This repeats until the best prompt is clear. Common methods include UCB and Successive Rejects.
will there be an intermediate layer between user prompt and input to LLM where we take the user prompt as is and then apply the prompt optimization technique before feeding it to LLM for final output ?	Typically, no. Full Prompt Optimization (like DSPy or ProTeGi) is run offline at design time because executing iterative feedback loops takes minutes and thousands of tokens, making it too slow for live users. In production, developers do use an intermediate layer for Prompt Rewriting, where a fast, lightweight model intercepts the user's raw input and reformats it to fit exactly inside your pre-optimized prompt template before sending it to the final LLM.
If Gradient Optimizaton is also using LLM for gradient or scoring then how this is different from Instruction search?	Both use LLMs, but they create new prompts differently. Instruction search (APE) generates many variations and keeps the high scorers, without knowing why a prompt failed. Gradient optimization first looks at the wrong answers, writes down why the prompt failed, and then fixes that specific flaw. So it's more targeted.

can you give an example of to calculate gradient based prompt	Take a prompt is "Classify the review as Positive or Negative." It fails on "Great, another phone that dies in 2 hours." The model says Positive, but the correct label is Negative. We ask the LLM: "Here is the prompt and the case it got wrong. Why did it fail?" The gradient it writes: "The prompt doesn't handle sarcasm." The new prompt: "Classify the review as Positive or Negative. Watch for sarcasm, where positive words express a negative meaning."
Can I assign weightage to the prompts as well as apart from scoring ? like if any output generated by a prompt it should always be considered irrespect of the score to avoid any error in generating the result ? and in same manner if any the output of a prompt is incorrect or not desired then just reset it	Yes, you can assign weights to evaluation criteria or prompt candidates, not just use a single overall score. For critical requirements, use hard constraints/validation rules and if the output violates them, reject or regenerate it regardless of its score. You can also define a reset/fallback mechanism when an output is invalid, while acceptable outputs can proceed even if their numerical score is not the highest.

If a trained LLM model is good in getting a semantic meaning. What are the reasons that during a jail break attack that LLM is not able to get the semantic meaning of that prompt like in example of axe murderer?	The model does understand the words, but the attack hides its intent inside a harmless-looking frame, like a conversation between two people. The original prompt told the model to look for an attempt to break protections, so it only checked for direct attempts and missed the hidden one. Models also tend to follow the story or role-play framing. That's why the gradient fixed the prompt by adding "regardless of how subtle or indirect."
what are the scenarios, where using the same model for LLM as judge or LLS as Evaluator makes no difference..or it is always advised to use a different model to stop biasness ?	It is acceptable to use the same model when the evaluation is strictly objective, such as checking for valid JSON formatting, word counts, or the presence of specific keywords. but, for subjective metrics like tone, reasoning quality, or helpfulness, models suffer from self-enhancement bias and will unfairly prefer their own outputs. Industry best practice is to always use a different, usually larger model (like using GPT-4 to judge a smaller open-source model) when evaluating qualitative traits.
is the prompt optimization is also cost effective	It costs extra at the start, since optimization runs many LLM calls. It usually pays off later, because the better prompt is reused on every user request. It can cut errors and retries, shorten prompts, and even let a smaller, cheaper model do the job. It's most cost effective for high-volume apps and less so for one-off tasks.

can we change the temperature of a defined model by passing the temperature in prompts, example GPT ?	No, parameters like temperature cannot be modified by passing it in the prompt. It has to be set with the API parameters during API call. In the web interface, it is not accessible and is set by the company.
What is GAN?	A GAN (Generative Adversarial Network) is a model where two networks compete: a generator creates fake data, and a discriminator tries to tell fake from real, so both keep improving until the fakes look real. In prompt optimization, the same idea is used when one LLM writes prompts and another critiques or tries to catch their failures.

In most real world use cases, prompts are usually 100s of lines with system instructions, examples and dynamic content combined (either in the prompt or during inferencing). How do you run the gradient approach in this setup? My assumption is we go through the following 2 phases - Phase-1 : In Phase-1 you evaluate which structure of the prompt works best based on the o/p score from the gradient APO setup. i.e prompt-1 with system instruction, examples, context prompt-2 with system instruction, examples, context, CoT etc Phase-2: ONce you have fixed on the structure of the prompt, then you build different versions of the prompt by changing the sentence structure, syntax, examples provided etc. Is this a fair assumption to make? Or is there a better approach?	Yes, that's a fair approach: fix the structure first, then refine the wording.
if hyper parameters k p temp are updated, still output is deterministic then is it not good thing? how or what scenario become problem like mode collapse	Being consistent is good when there's one right answer, like classification, extraction, or math. It becomes a problem when you need variety. If outputs stay the same even after raising temperature or top p, that's mode collapse. For example, in APO the optimizer keeps generating near-identical candidate prompts, so the search gets stuck. It also hurts brainstorming, creative writing, and synthetic data generation, where you keep getting the same few ideas.

But if I want to create my own agent using any Model in background like KiMi K2 I can set the temperature in that agent right ?	Yes, if you are using API call then you can set the parameters
what is Speculative decoding? as we understand this is opposite to autoregressive generation, what ever optimizations we are learning here will also apply for Speculative decoding as well?	Speculative decoding is not the opposite of autoregressive generation, it is an inference acceleration technique that produces the exact same autoregressive output much faster. A smaller, faster model proposes candidate tokens, and the primary model verifies them in parallel in a single forward pass. Because it is purely an invisible backend serving optimization that does not alter output quality or behavior, every prompt engineering and optimization technique (like DSPy or ProTeGi) applies identically.
What if I don't like the response generated in joke question, will the probability goes down by recording my feedback ?	Not right away. Your thumbs-down doesn't change the model live; the company may collect it and use it later to train future versions (RLHF), which might make such jokes less likely then. To get a different joke now, ask directly, such as "give me a different joke, not a pun," or raise the temperature for more variety.
so for verbalized sampling are we sort of selecting ourselves i.e. human?	Not necessarily. The model lists the responses with probabilities, and then code usually picks one at random using those probabilities (Step 3). You can also keep all 5 for variety. A human can choose if they want, for example in creative work, but it's not required.

i got 5 diffrent responses while asking "jokes on coffee" *5 to LLM	That's a great observation! Mode collapse is a tendency, not a rule. If you asked in the same chat, the model can see its earlier jokes and avoids repeating them. Chat apps may also use higher temperature, and newer models are better at variety. The slide's result comes from asking the same prompt in separate, fresh calls. Try that and you'll likely see more repeats.
In this slide when we are prompting 5 times then are we using the same Model always or using different AI Tools?	Same model every time. In the direct prompting row, the same prompt went to one model 5 times, and it kept producing similar desert-style captions. In the Verbalised Sampling row, the same model was asked once for 5 different captions with probabilities. Only the prompting method changed, not the model or tool.
Hoe does verbalized sampling avoids hallucinations ? as diversity increases so lower prob. things will also be picked .	Good point! Verbalised Sampling isn't meant to prevent hallucinations. It's for tasks where many answers are valid, like jokes or stories, so a lower-probability option is still a good answer, just a less common one. For factual tasks with one correct answer, don't use it for variety. If you do use it, you can ask only for responses above a set probability, or verify the facts before using them.
we have seen prompt optimisation, but if my app features a chat option. can we optimize the given by the end-user?	Yes. A common way is query rewriting: before the main model answers, a quick, cheap LLM call rewrites the user's message into a clearer one. It fixes typos, adds context from the chat history, and makes the request specific. If the message is too vague, the app can ask a clarifying question instead. Just keep in mind that the extra step adds a little cost and delay.

where the prompt optimization specifically used? For code generate if we use first prompt optimization and then use the optimized prompt then we get good output and reduced token ?	It's used mostly in apps that run the same kind of prompt many times, like chatbots, classification, data extraction, summarization, and coding assistants. For code generation, yes: an optimized prompt, tested on sample coding tasks, usually gives more correct code with fewer retries and shorter, focused output, which saves tokens. For example, adding "return only the Python function, no explanation" cuts extra text and makes the output easier to test.
So, overall for the problem of mode collapse, best solution is Verbalized sampling but Temperature with top p can help to some extent as well. Is the understanding correct?	Yes, that's right! Temperature and top p help a little, but as the slide showed, outputs often still come out similar. Verbalised Sampling works much better for this. The two can also be used together, since Verbalised Sampling combines with temperature and top p.
can you explain the difference between likelihood of Instruction search v/s gradient of Gradient Optimization? Both uses LLM for scoring.	Likelihood (instruction search) is a number. It shows how likely the model is to give the correct output with a prompt, and it's used only to rank prompts. It tells you how good a prompt is, not why. A gradient optimization is written feedback on why the prompt failed on wrong examples, like "misses subtle attacks." It's used to rewrite the prompt, not to score it. The new prompts are then scored separately, for example by accuracy.

In that case, how gradient optimization handles the subjectivity when "Why" can vary? Does it give better prompt along with "Why"?	To handle subjective or varying reasons for failure, frameworks like ProTeGi process errors in batches and ask the LLM to generate multiple different critiques simultaneously. Yes, it also generates the better prompt, the framework first uses an LLM to generate the "Why" (the critique), and then uses a second LLM call to automatically rewrite the original prompt based on that feedback. Finally, it tests all the new variations and selects the one that objectively scores the highest.
I have llm of 120B parameter for FAQ search (from company FAQ documents). now as end customer I am asking for FAQ's, but response is still hallucinating (suppose RAG implemented), what could be the probable mistake I did as AI developer and where we can improve	Even with a massive 120B model, RAG systems hallucinate when the retrieval pipeline feeds the model fragmented, irrelevant, or poorly parsed context. As a developer, the most common mistakes are improper document parsing (like scrambling tables), bad chunking boundaries that split FAQ answers in half, or relying on weak embeddings that retrieve the wrong documents. To improve, fix your data pipeline by ensuring Q&A pairs are chunked cohesively, implement hybrid search (semantic plus keyword), and add strict system prompts instructing the model to refuse to answer if the retrieved context is insufficient.

what happens if a role playing prompt has another role assumption, like multi layer of role. example: You are an actor playing a detective who is undercover as a chef How does LLM suppose to behave?	The model usually plays the innermost role on the surface (the chef), while keeping the outer layers as background. So it talks like a chef but may drop detective hints. With many layers, it can get confused about which role should answer and switch between them. To keep behavior consistent, clearly state which role speaks and when. Nested roles are also a common jailbreak trick, so the model's safety rules still apply at every layer.
Can you give a sample prompt for the other LLM to evaluate	"You are an evaluator. Given the question, the answer, and the reference answer, rate the answer from 1 to 5 for correctness, where 5 means fully correct and matches the reference, and 1 means wrong or made up. Reply in JSON as {"score": number, "reason": "one line"}. Question: {question} Answer: {llm_output} Reference: {ground_truth}" The app then checks whether the score passes the threshold, for example 4 or above.
Is there a limit from the LLM models, how many tools can be called for a single prompt?	It depends on the provider. There's usually a cap on how many tools you can define in one request (for example, OpenAI allows up to 128). The model can call several tools per turn, and in an agent loop it can keep calling tools until the context window fills or your app's step limit stops it. Note that in practice, accuracy drops when there are too many tools, so it's best to give only the ones the task needs.

If the Judge LLM is just another error-prone language model that can make mistakes, how can we trust it to grade another model?	That's a fair concern. We don't trust it blindly, we check it. First, compare the judge's scores with human ratings on a sample, and trust it only if they mostly agree. To make it more reliable, give it a clear rubric and reference answers, ask for a reason with each score, use a strong or different model, and average several judges or runs. Humans should still spot-check results regularly.
if we get low values for NLI Score or BLEURT then do change the weights or increase the temperature?	Usually neither. A low score means the output doesn't match the reference, so first improve the prompt (clearer instructions, examples, better context) or the retrieved context. Raising the temperature usually makes it worse, so lower it for factual tasks. Changing weights means fine-tuning, which is a last resort. Also check that the reference answers themselves are correct.
are these scorers meant to be used before the LLM is used in production for a specific task or after for optimization?	Both. Before production, they're used to test and compare prompts or models on a test set and choose the best one. After launch, they're used to monitor live outputs, catch drops in quality, and guide further optimization.
for this eval is it need anther LLM or same llm will work	Either works, but using a different, strong model is better. Coherence is a subjective judgment, and the same model may rate its own summaries too kindly. The original G-Eval paper used GPT-4 as the judge.
As prompts and LLMs are tightly encapsulated,so while designing the systems in agentic AI ..should it have both prompt evaluation and LLM evalautaion techniques as well	Yes, ideally both. Evaluate the LLM to pick the right model, and evaluate the prompts to tune each agent's instructions. For agents, also check agent-level metrics like task completion and tool correctness, since a good model with good prompts can still pick the wrong tool or take wrong steps.

what is the criteria to select LLM B which will evaluate LLM A ? how many LLM evaulators one should keep in production ? Will it evaluate each and every response that LLM A is generating and then rate it or we schedule and get the result ?	To select your Judge (LLM B), choose a frontier model with significantly higher reasoning capabilities than your production model (LLM A), such as using GPT-4 or Claude 3.5 Sonnet to evaluate a smaller 8B model. In production, you rarely evaluate 100% of responses with a heavy judge because it doubles your latency and costs; instead, you run it asynchronously on a scheduled, statistical sample of your logs (5-10%). If you must evaluate 100% of traffic in real-time, you deploy tiny, fast models that act as lightweight circuit breakers for objective rules rather than full subjective judges.
like local llm ollama any evaulation llm available to use?	Yes
which model is genenarting CoT? LLM A or LLM B?	LLM B, the judge. It creates the evaluation steps (the CoT) from the task and criteria, then uses them to score. LLM A only produced the summary being evaluated.
Will the chain of thought generated from LLM B for evaluation ?	Yes. LLM B generates the evaluation steps (CoT) once, from the task and criteria. The same steps are then reused in the scoring prompt for every output being evaluated, so all outputs are judged in the same way.
Based on the experiment, do we also check the evaluation prompt that was geenrated using CoT ? may that might mislead the coherence scores?	Yes, and it's good practice to do so. Since every score depends on those generated steps, a human should read them first, and they can be edited or written by hand if needed. Then test the setup by comparing G-Eval's scores with human ratings on a sample. The G-Eval paper did exactly this, and if the scores don't match human ratings well, you fix the steps.

For G-Eval, do we need ground truth?	No, G-Eval doesn't need a ground truth answer. It scores the output using the criteria and the input, like judging a summary against its article for coherence. You can add a reference answer if your criteria need it, such as correctness. Human ratings are useful only when checking how reliable the judge is.
Why cannot these BLEU, ROUGE, and METEOR metrics alone guarantee the factual accuracy of AI-generated text?	BLEU, ROUGE, and METEOR mainly measure surface-level similarity between generated text and a reference text, such as shared words, phrases, or linguistic overlap. A response can have high scores while still containing factually incorrect information, and a correct response may get a low score if it uses different wording.
Prompt compression will also compress the context. Is this correct?	Yes. In practical enterprise architecture, compressing the context (such as retrieved documents in RAG or long conversation histories) is actually the primary focus and highest-yield target of prompt compression. Because a complete LLM prompt consists of instructions, examples, and dynamic context combined, frameworks like LLMingua specifically target the context portion to strip out irrelevant sentences and redundant words, reducing the token count by 20% to 80% while preserving the core factual meaning.

What does context means? What is called context memory?	Context refers to the entirety of the information provided to the model in a single request, including the system instructions, the current user query, retrieved documents, and past conversation history. Context memory (often called contextual memory or working memory) is the temporary, finite space where this information is actively stored and processed during inference. It is limited by the model's context window, meaning the model can only reason over and remember the tokens currently active in that space before it begins to drop earlier parts of the exchange.
Are there any tools available in market for compression techniques ? We can use these tools before sending prompts to LLM's	Yes. The best-known is Microsoft's LLMingua (and LLMingua-2), which is open source and works with LangChain and LlamaIndex; it uses a small model to drop unimportant words before the prompt is sent to the LLM. Another option is Selective Context, and some RAG tools include built-in context compressors. For example, LLMingua can shrink a 2,000-token retrieved document to about 400 tokens while keeping the key facts.
Is compression automated or human driven	It can be both. Manual compression is when a person rewrites the prompt to be shorter, removing repeated or unnecessary instructions, which is best for fixed system prompts. Automated compression uses tools like LLMingua to shorten long or changing inputs, such as retrieved documents, on every call.

What was the 2nd level of optimization for prompt compression	If this refers to the LLMingua-style method, compression happens in two levels. The first level (coarse) removes whole less-useful parts, like extra examples or sentences. The second level (fine) removes individual low-importance words or tokens from what remains. If the instructor meant something else, please share that slide and I'll adjust.
Do we need to Train the LLM with abbreviations prior before we use them in Token level optimisation?	No, we generally do not need to retrain the LLM before using abbreviations in token-level prompt optimization. The tokenizer can process abbreviations, but the model may understand uncommon abbreviations poorly, which can hurt performance. So we use standard/domain-specific abbreviations that the model already understands; for new abbreviations, define them once in the prompt or evaluate their effect experimentally
is LLMingua internal implements all these compression technique	Yes. LLMingua is an open-source library from Microsoft that does all of this automatically. A small language model scores how important each token is, then the low-value tokens are removed to create the compressed prompt. The family includes LLMingua, LongLLMingua (for long contexts and RAG), and LLMingua-2 (faster, using a trained classifier).
Is LLMingua opensource or requires licensing?	Yes. LLMingua is an open-source library from Microsoft that does all of this automatically. A small language model scores how important each token is, then the low-value tokens are removed to create the compressed prompt. The family includes LLMingua, LongLLMingua (for long contexts and RAG), and LLMingua-2 (faster, using a trained classifier).

Prompt compression can sometimes change or distort the original meaning of a prompt. How can we minimize the risk of misleading or losing important information during prompt compression, especially when dealing with very large prompts?	To minimize information loss when compressing large prompts (such as heavy RAG contexts), you should use Query-Aware Compression rather than generic summarization. This technique evaluates the relevance of every token or sentence directly against the user's specific question, aggressively deleting unrelated noise while preserving the exact data needed for the answer. also, You should implement Dynamic Budget Allocation, applying heavy compression to low-relevance documents while leaving highly relevant chunks largely intact, and strictly protect structural elements by ensuring your system instructions, few-shot examples, and the final user query bypass the compressor entirely.
how actula token 778	We used a tokenizer to tokenize the text and the token count came to 778

is it suggested to apply compression for every prompt in the session or apply it after reaching a certain context limit? is it similar to compacting?	Prompt compression and context compaction are distinctly different techniques with different trigger mechanisms. Prompt compression (removing redundant words or noise) is typically applied to every prompt's dynamic context, such as retrieved RAG documents, to reduce latency and cost continuously. Context compaction is a session management strategy that is only triggered after reaching a specific token threshold, usually around 70-75% of the model's context limit.
Prompt compression 1. is it comparable to sending vectors - using BoW or One-Hot encoding? 2. will this not be an issue for context? Ex - customer comments sarcastically, and the response is a thank you - happened with Indigo	Prompt compression is not the same as sending vectors using BoW or one-hot encoding. Compression still sends a shortened textual prompt, whereas BoW/one-hot converts text into numerical representations that are not directly equivalent to the original language input. Yes, compression can cause contextual errors, especially with sarcasm, negation, tone, or subtle cues. If the compressor removes words like “great”, “really?”, or surrounding context, the model may interpret a sarcastic complaint as genuine praise—leading to an inappropriate “thank you” response.

Would it be better to split a large prompt into multiple smaller sub-prompts or instruction modules instead of compressing everything into one prompt? If so, how can we determine which sub-prompts are relevant and need to be included for a particular user request?	Often yes. Keeping a small core prompt that's always included, plus separate modules added only when needed, is a common and effective approach. To choose the right modules for each request: - Use a router: a small LLM or classifier finds the user's intent and picks the matching modules. - Use simple rules or keywords for clear cases. - Use embeddings to fetch the most similar modules, like in RAG. Also test the router, since a wrong choice means missing instructions.
is this how caveman works?	If you mean the "caveman" trick of telling the LLM to write in very short, caveman-style language, it's a similar idea (drop filler words) but works differently. Caveman is just an instruction that makes the LLM itself write shorter, cutting mostly output tokens. LLMLingua uses a separate small model to shrink the input prompt before it's sent to the LLM.
if we compress context are we forcing minimizing the context?	Yes, compression shrinks the context, but the goal is to remove only the unimportant parts, like filler words, repetition, and irrelevant sections, while keeping the key information. If you compress too hard, useful details get lost and answer quality drops, so you test to find the right level.
Is there any cost involved to use the LLMLingua? if yes how the payment calculated is it pay as you use?	No license fee. LLMLingua is free, open-source software from Microsoft. The only cost is running its small model on your own computer or cloud server (CPU or GPU), which you pay for like any other compute. There's no pay-per-use charge, and it usually saves money by cutting the tokens you pay for on the main LLM.

What is black box LLM?	Black box LLM means the internal parameters of the model are not accessible to use
Does the iterations also add to Token usage? How can efficiency be attained if the process to compress the prompt consumes more tokens?	The compression is done by a small model running on your own machine, not by the paid LLM, so it doesn't add to your API token bill. It only uses a little compute. LLMingua-2 is even lighter, since it's a small classifier that works in one pass. The big saving comes from the large LLM reading far fewer tokens, like 2366 cut to 117. Fixed parts, like examples, can also be compressed once and reused.
while using llmlingua how do we know that these words are not required and doesnt make any sense in the context?	LLMLingua uses a small language model to judge each word: words that are easy to predict from the surrounding text, like "the," "is," or repeated phrases, carry little new information and get dropped, while surprising or key words are kept. LLMLingua-2 instead uses a trained classifier that marks each word as keep or drop.
is LLM Lingua a trained model or an algorithm	LLMLingua uses a small language model to judge each word: words that are easy to predict from the surrounding text, like "the," "is," or repeated phrases, carry little new information and get dropped, while surprising or key words are kept. LLMLingua-2 instead uses a trained classifier that marks each word as keep or drop.

can you provide a practical example where this is being used?	A common use is in RAG chatbots: a company's support bot retrieves 10 long policy chunks for a question, LLMingua compresses them to their key sentences, and only that shorter text goes to the LLM, cutting cost and response time. It plugs directly into LangChain and LlamaIndex for this. Another example is summarizing long meeting transcripts, where LLMingua-2 strips filler talk like "um, so, you know" before the transcript is sent for summarization.
is it a frontier model?	No. LLMingua-2 uses a small encoder model like based on XLM-RoBERTa or multilingual BERT, with only a few hundred million parameters, far smaller than frontier models. GPT-4, a frontier model, was used only once to create its training data.
which one should we use LLMingua-1 or LLMingua-2 considering these two are compressing the input differently? and under what scenarios we should use which?	LLMingua-2 is usually the default choice. It's much faster, uses a small classifier, and works well on general text like meeting notes, documents, and summaries without depending on the task. LLMingua-1 helps when the prompt has many few-shot or step-by-step examples, like math reasoning, because its budget controller can drop whole examples first. For RAG with long retrieved documents, use LongLLMingua, which keeps the parts relevant to the question. Test both on your own data before choosing.
in lingua2, do we generate the training model for every new prompt? isn't it not efficient?	No. LLMingua-2's classifier is trained once by Microsoft and then reused on any new prompt, so you only run it, you don't retrain it.

How latency is reduced? Best ways to do so?	The LLM spends time reading every input token before it starts answering, so fewer tokens means a faster first response. Best ways to cut latency: - Shorten or compress the input. - Limit output length, since writing tokens is the slowest part. - Use a smaller, faster model for simple tasks. - Use prompt caching for repeated parts. - Stream the answer and run independent calls in parallel. - Keep chains to as few steps as needed.
why is it that lingua and lingua2 evaluate using different formula?	They were built with different ideas. LLMingua-1 uses perplexity: a small language model reads left to right, and words it can easily guess are treated as less important. It needs no extra training, but it only sees the words before each one. LLMingua-2 treats compression as a keep-or-drop classification, learned from GPT-4 examples. It sees the whole sentence at once, which makes it faster and more accurate at keeping meaning.
Is LLMingua2 pretrained for enterprises to use?	Yes. Microsoft has released ready-to-use pretrained LLMingua-2 models on Hugging Face under an open-source (MIT) license, which allows commercial use.
Apart from LLMingua, do we have any other prompt compression/Optimization LLMS?	Yes. For compression - RECOMP. For optimization DSPy, Microsoft's PromptWizard.

What language model we use for LLMlingua1 and 2	LLMLingua-1 uses a small causal language model to score words. The library default is Llama-2-7B, and smaller ones like GPT-2 or Phi-2 can also be used. LLMLingua-2 uses an encoder model, XLM-RoBERTa-large (the one you will see in today's notebook: microsoft/llmlingua-2-xlm-roberta-large-meetingbank)
thankyou for the example. So basically it supports where we use bots or agents and we need to optimize the cost (probably the biggest challenge). Rather than LLM going through large chunk, compress it and then let it ask LLM's.	Yes, compression helps in reducing token cost and latency for the downstream LLM.
For reasoning models; what is the way to decrease token consumption, Do the intermediary thinking steps consume tokens?	Yes, intermediary thinking steps absolutely consume tokens. When you use reasoning models, the internal chain-of-thought is generated as reasoning tokens. While these tokens are often hidden from the user interface, they are billed directly to your account at the standard output token rate, which can make reasoning queries significantly more expensive than standard LLM queries.
I still dont get how latency is reduced, since rather than LLM understanding the entire text, the compression layer is taken up that time which was suppose to be saved by a smaller prompt	It saves time only because the compressor is much smaller and faster than the main LLM. LLMLingua-2 reads the text once with a small model, which takes a fraction of a second. The big LLM, with billions of parameters, spends much more time on every token it reads. So "quick compression + big LLM on a short prompt" usually beats "big LLM on a long prompt." You're right, though, that for short prompts or a small main LLM, the saving can be tiny or even negative, so measure it for your case.

will LLMingua2 also work effectively on inputs involving Code (Programming Language) . Eg on C Code which is not like plain english comparing to python	Not very well. LLMingua-2 was trained on natural language (meeting transcripts), and code needs every symbol. Dropping a bracket, semicolon, or keyword can change or break the logic, and that's even riskier in C than in Python. For code, it's better to leave the code itself uncompressed and instead remove comments and extra whitespace, send only the relevant functions or files, or give short summaries of the rest.
What are the safety guards that can applied in LLM to prevent these attacks	Based on today's examples: - Keep secrets out of the system prompt. In the FinSecure case, the internal URL leaked because it was in the prompt. - Keep system instructions and user or external content (like the resume) clearly separated, and tell the model to treat outside content as data, not commands. - Check inputs for phrases like "ignore all previous instructions" before they reach the model. - Use an LLM judge to check outputs, blocking leaked instructions or PII like account numbers. - Add known attacks to your regression test set, and rerun it on every new prompt version.
Do we have to train LLMingua-2 or rather can we train it for our specific case. Like a lawyer might want every minimal detail and some words may entirely change the meaning of the clause. Can we add our layer above LLMingua2?	You do not have to train LLMingua-2, but you absolutely can fine-tune it for specific domains, and you can definitely add a rule-based protection layer on top of it. LLMingua-2 is designed as a task-agnostic model (trained via data distillation from GPT-4), meaning it works effectively out-of-the-box for general use cases.

Is there a way to identify which parts of the prompt are given a higher weightage? specially for long prompts. Ex - If I use NER on complex legal documents, I have hierarchial conditions. How can I evaluate weightage for the instructions given - or may be if it is completely ignored?	There's no direct way to see weights, but you can use today's evaluation workflow: - Versioning and regression testing: make a new version with one instruction removed or changed, and rerun your test set. If results don't change, that instruction is being ignored. - LLM as a judge with a rubric: give one point per condition, like the loan notebook, so you see exactly which rules are followed or missed. - Textual gradients (ProTeGi): the judge's feedback explains which instructions the model skipped and why. For deep hierarchies, prompt chaining helps by handling one level of conditions per step.
--	--

For this banking related injection issue will the LLM not use users access level to query the DB which must have some RBACs applied? Rather can we setup the agent system in a way so that the session runs on user context or access level granted for a customer/user and not any generic service principles that exposes entire DB.	By default, an LLM has no intrinsic awareness of access levels, if connected using a generic, high-privilege service principal, a successful prompt injection will expose unauthorized database records Your proposed solution is the exact enterprise industry standard, you should never grant the agent direct, sweeping database access. Instead, enforce Identity Delegation and Row-Level Security (RLS) by running the agent's tool execution strictly within the authenticated user's session context (via OAuth 2.0 On-Behalf-Of tokens or session-bound parameters). This ensures that even if prompt injection completely subverts the LLM's logic, the database engine itself enforces the access boundary and physically denies retrieval of any unauthorized customer data.
In LLM Lingua2 if we notice that the compression is leading to loss of some context . Can we place a filter that if LLM Lingua sees certain keywords it does not compresses that prompt . like an exceptions rule .	Yes. LLM Lingua-2's compress_prompt has a force_tokens option, a list of words or symbols that are always kept, like force_tokens=["revenue", "guidance", "\$"]. There's also force_reserve_digit=True to keep all numbers, which would have helped in today's notebook, where some guidance figures were lost. For a full exception rule, add a simple check in your code: if the text contains certain keywords, skip compression and send the original prompt.

does Context memory hold sequence of compressed prompts at any given time OR summarized overall context at any given time OR unmodified user and bot interaction?	Context memory does not have any single format. it holds exactly what the AI developer programs the application to pass into the API for that specific turn. Depending on the application's memory architecture, context memory can utilize any of the formats you mentioned, though the first two are the most common Unmodified User and Bot Interaction Summarized Overall Context (Summary Buffer) Sequence of Compressed Prompts
Please give the link here....it enables copying the link	Notebook link?https://colab.research.google.com/drive/1R2P112ZfZQRQZfyY09jCS5tAZjUyW_a?usp=sharing

Do we have any option to compress image prompts which are sent to Vision Models	LLMLingua works only on text, but images can be shrunk too. The simplest ways are to resize or lower the image resolution and crop to just the part that matters, since vision models charge tokens based on image size. Some APIs also offer a low-detail setting. If only the text in the image matters, run OCR first and send the text instead. Compression isn't always the best choice. Often it's better to send less in the first place: retrieve only the relevant parts (as in RAG), summarize older context, write tighter prompts through manual optimization, or use prompt caching for repeated parts. Compression helps most with long, wordy text that has to be sent. As today's notebook showed, the ROUGE-L score dropped from 0.38 to 0.23 after compression, so always test before choosing it.
How the models are chosen?	The same way we choose prompts: test candidate models on your own test set with a metric, like accuracy, an LLM judge, or ROUGE, and compare. Also weigh cost, speed, context length, language support, and whether data must stay on your own servers. For the compressor, LLMLingua-2 is usually the default for speed, with LLMLingua-1 when prompts have many examples.

I am using Claude LLM and I want to use the token compression using LLMingua, How I can do that?

It works the same way as today's notebook, with Claude in place of Llama. Install `llmlingua`, load `PromptCompressor` with the `llmlingua-2` model, and run `compress_prompt` on your long context. Then add your instruction back and send the compressed prompt to Claude through the Anthropic API (the anthropic Python library, `messages.create`). LLMingua runs on your own machine, so Claude only receives and bills the shorter prompt.

I am using Claude LLM and I want to use the token compression using LLMLingua, How I can do that?

```
!pip -q install anthropic llmlingua

from anthropic import Anthropic
from llmlingua import PromptCompressor

client = Anthropic(api_key="YOUR_API_KEY")

compressor = PromptCompressor(
    model_name="microsoft/llmlingua-2-xlm-roberta-large-
meetingbank",
    use_llmlingua2=True
)

def generate_answer(prompt):
    response = client.messages.create(
        model="claude-sonnet-5",
        max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text

compressed = compressor.compress_prompt(context, rate=0.3)
compressed_prompt = compressed["compressed_prompt"] +
"\n\n" + question

answer = generate_answer(compressed_prompt)
print(answer)
```

I am using Claude LLM and I want to use the token compression using LLMLingua, How I can do that?	The Llama tokenizer won't match Claude's token counts, so for token counting use <code>client.messages.count_tokens</code> or the <code>origin_tokens</code> and <code>compressed_tokens</code> values that <code>compress_prompt</code> returns.
why -1	In <code>response[0]["generated_text"][-1]["content"]`</code> , the <code>`generated_text`</code> holds the whole conversation as a list: system, user, then assistant. The index <code>`-1`</code> in Python means the last item, which is the assistant's reply, so it picks out just the model's answer.
can we count the tokens for closed weight model?	Yes. Even without the model weights, providers let you count tokens. OpenAI has the <code>tiktoken</code> library, Anthropic has a <code>count_tokens</code> API, and Gemini has a <code>count_tokens</code> method. Every API response also reports the exact input and output tokens used in its <code>usage</code> field.
why are we taking assistant content in the response?	Because <code>generated_text</code> returns the whole chat, including our own system and user messages. Those are just our input echoed back. The only new part written by the model is the assistant message, so we take its content as the answer.
when should we use <code>add_special_tokens</code>	In the notebook we used <code>`add_special_tokens=False`</code> because we only wanted to count the tokens in our text. By default, the tokenizer adds special tokens like the begin-of-text marker, which would slightly inflate the count. Keep it <code>True</code> (the default) when you tokenize a full input to feed directly to the model. When using the pipeline with chat messages, special tokens are handled automatically.

why we need dataset here	We need the dataset for two things. First, it gives us a real, long earnings call transcript to compress, which is where compression helps most. Second, it includes a human-written summary we use as the reference to score both answers with ROUGE-L, so we can check whether compression kept the important information.
When we should use LLMlinga	Use LLMlinga when your prompts carry long, wordy context, like meeting or call transcripts, long documents, retrieved RAG passages, or many examples, and you want to cut cost, speed up responses, or fit within the context limit. Avoid it for short prompts, code, or text where every exact word or number matters. Always test quality before and after, as we did with ROUGE-L in today's notebook.
Where to access the notebook for hands-on	https://colab.research.google.com/drive/1R2P112ZfZQRQZfgY09jCS5tAZjUyW_a?usp=sharing
how to proactively identify and fix prompt injection (may be html tags/utl/sql etc)) promptly before it is caught on gateway as WAF rule	Check inputs in your own app before they reach the LLM: strip or escape HTML and scripts, allow only trusted URLs, and use a small injection-detection classifier (like Meta's Prompt Guard) to flag lines such as "ignore previous instructions." Wrap user and retrieved content in clear tags so the model treats it as data, not commands, and never let the LLM run raw SQL.

How can we design our domain specific compression technique	You can follow the same 5 steps as the LLMingua-2 slide, using your own domain texts: 1. Have a strong LLM compress sample documents, keeping key domain facts. 2. Label each word as keep or drop. 3. Filter out poor examples. 4. Fine-tune a small classifier like XLM-RoBERTa on these labels. The LLMingua repo has training scripts. 5. Use it to compress new prompts. Before training, first try force_tokens with your key domain terms, which may already be enough. Always evaluate on a domain test set.
--	--